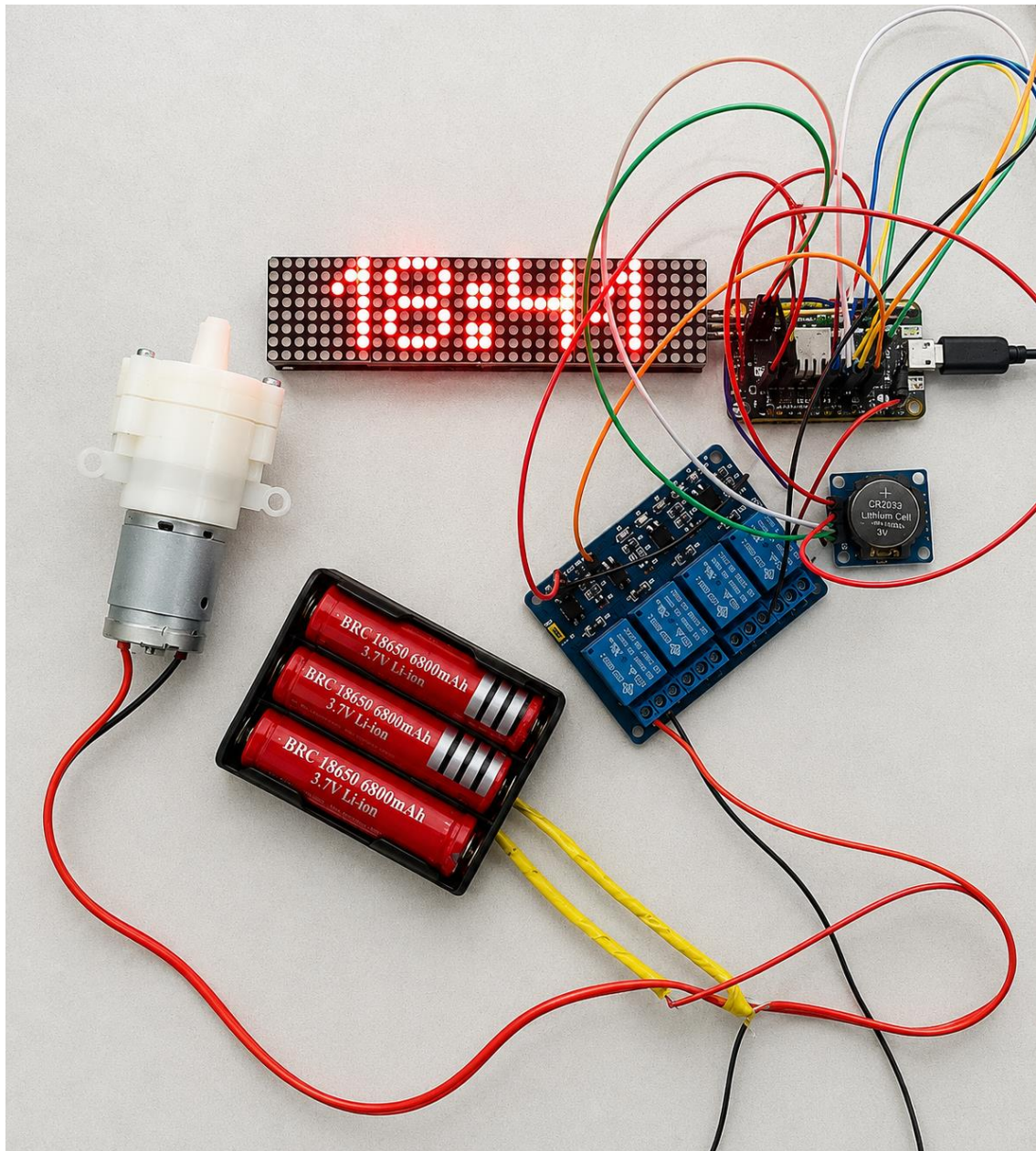


Smart Pump

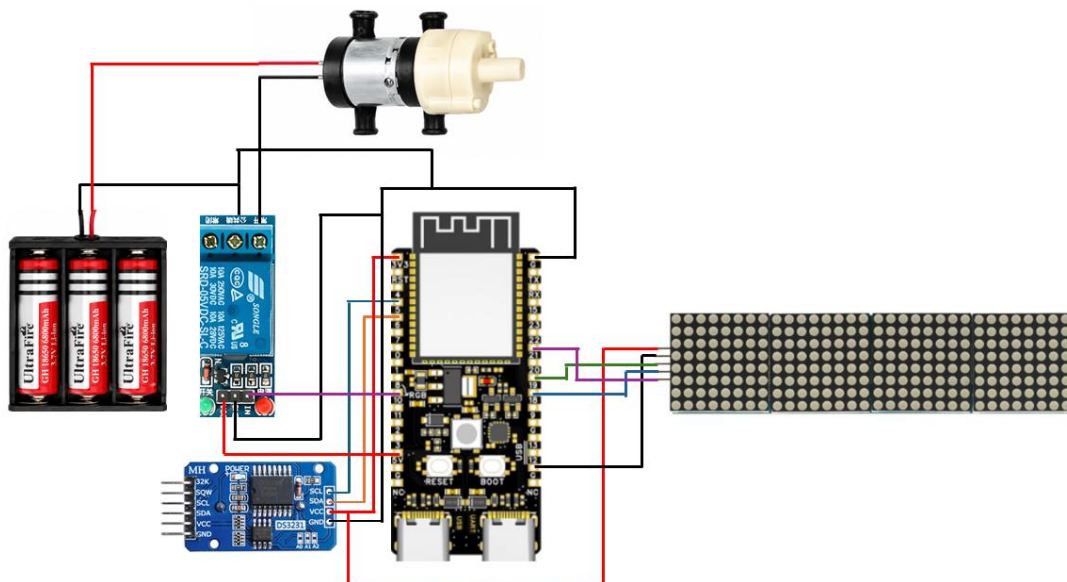


Aim: To Develop an IoT-based Smart Pump System using ESP32C6, RTC (DS3231), Relay Module, and Water Pump that automatically controls water pumping operations based on user-defined schedules while displaying real-time information on a MAX7219 Dot Matrix Display.

Components Required:

Sno	Component Name	Quantity
1	ESP32C6 Dev Module	1
2	12V Pump Motor	1
3	5V Relay	1
4	RTC (DS3231) Module	1
5	Max7219 panel	1
6	3x18650 battery holder	1
7	18650 battery cell	3
8	Jumper Wires	As required

Circuit Diagram:



Procedure:

Detailed Wiring Connections

1. Battery Pack Connections

1. Connect the **Battery Positive (+)** wire to the **COM terminal** of the relay module.
2. Connect the **Battery Negative (-)** wire to the **negative terminal of the water pump**.
3. Connect the **Battery Negative (-)** wire to the **GND terminal of the relay module**.
4. Connect the **Battery Negative (-)** wire to the **GND pin of the ESP32**.

2. Water Pump Connections

1. Connect the **Positive terminal (Red wire)** of the water pump to the **NO (Normally Open)** terminal of the relay module.
2. Connect the **Negative terminal (Black wire)** of the water pump to the **Battery Negative (-)** terminal.

3. Relay Module Connections

Control Side

1. Connect **VCC** of the relay module to the **5V pin of the ESP32**.
2. Connect **GND** of the relay module to the **GND pin of the ESP32**.
3. Connect **IN** of the relay module to **GPIO 10** of the ESP32.

Switching Side

1. Connect **COM** terminal to **Battery Positive (+)**.
2. Connect **NO** terminal to the **Positive terminal of the water pump**.

4. DS3231 RTC Module Connections

1. Connect **VCC** of the DS3231 module to **3.3V** of the ESP32.

2. Connect **GND** of the DS3231 module to **GND** of the ESP32.
3. Connect **SDA** of the DS3231 module to **GPIO 21** of the ESP32.
4. Connect **SCL** of the DS3231 module to **GPIO 22** of the ESP32.

5. MAX7219 Dot Matrix Display Connections

1. Connect **VCC** of the MAX7219 module to **5V** of the ESP32.
2. Connect **GND** of the MAX7219 module to **GND** of the ESP32.
3. Connect **DIN** of the MAX7219 module to **GPIO 23** of the ESP32.
4. Connect **CS** of the MAX7219 module to **GPIO 5** of the ESP32.
5. Connect **CLK** of the MAX7219 module to **GPIO 18** of the ESP32.

6. ESP32 Power Connections

1. Power the ESP32 using a USB cable.
2. Ensure the following common connections:
 - ESP32 GND ↔ Relay GND
 - ESP32 GND ↔ RTC GND
 - ESP32 GND ↔ MAX7219 GND
 - ESP32 GND ↔ Battery Negative (-)

Full Code:

```
#define BLYNK_TEMPLATE_ID "ENTER_YOUR_TEMPLATE_ID"
#define BLYNK_TEMPLATE_NAME "ENTER_YOUR_TEMPLATE_NAME"
#define BLYNK_AUTH_TOKEN "ENTER_YOUR_AUTH_TOKEN"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>

#include <Wire.h>
#include <RTClib.h>
```

```
#include <MD_Parola.h>
#include <MD_MAX72xx.h>
#include <SPI.h>

char ssid[] = "ENTER_YOUR_SSID";
char pass[] = "ENTER_YOUR_PASSWORD";

// RTC
RTC_DS3231 rtc;

// MAX7219
#define HARDWARE_TYPE MD_MAX72XX::FC16_HW
#define MAX_DEVICES 4

#define DATA_PIN 19
#define CLK_PIN 21
#define CS_PIN 18

MD_Parola display =
MD_Parola(
  HARDWARE_TYPE,
  DATA_PIN,
  CLK_PIN,
  CS_PIN,
  MAX_DEVICES);

// Relay
#define RELAY_PIN 10

BlynkTimer timer;

// Schedule variables
int pumpOnHour = -1;
int pumpOnMinute = -1;

int pumpOffHour = -1;
int pumpOffMinute = -1;

bool pumpRunning = false;
```

```
// -----  
// Blynk Sync  
// -----  
  
BLYNK_CONNECTED()  
{  
  Blynk.syncVirtual(V0);  
  Blynk.syncVirtual(V1);  
  Blynk.syncVirtual(V2);  
  Blynk.syncVirtual(V3);  
}  
  
// -----  
// Receive ON Time  
// -----  
  
BLYNK_WRITE(V0)  
{  
  pumpOnHour = param.asInt();  
  
  Serial.print("ON Hour : ");  
  Serial.println(pumpOnHour);  
}  
  
BLYNK_WRITE(V1)  
{  
  pumpOnMinute = param.asInt();  
  
  Serial.print("ON Minute : ");  
  Serial.println(pumpOnMinute);  
}  
  
// -----  
// Receive OFF Time  
// -----  
  
BLYNK_WRITE(V2)  
{  
  pumpOffHour = param.asInt();
```

```
Serial.print("OFF Hour : ");
Serial.println(pumpOffHour);
}

BLYNK_WRITE(V3)
{
    pumpOffMinute = param.asInt();

    Serial.print("OFF Minute : ");
    Serial.println(pumpOffMinute);
}

// -----
// Clock Update
// -----

void updateClock()
{
    DateTime now = rtc.now();

    char timeBuffer[6];

    sprintf(timeBuffer,
            "%02d:%02d",
            now.hour(),
            now.minute());

    // MAX7219 Display
    display.displayClear();

    display.displayText(
        timeBuffer,
        PA_CENTER,
        0,
        0,
        PA_PRINT,
        PA_NO_EFFECT);

    display.displayAnimate();
}
```

```
// Blynk Time Display
Blynk.virtualWrite(V4, timeBuffer);

Serial.print("Current Time : ");
Serial.println(timeBuffer);

// -----
// Pump ON
// -----

if (pumpOnHour >= 0 &&
    pumpOnMinute >= 0 &&
    now.hour() == pumpOnHour &&
    now.minute() == pumpOnMinute &&
    !pumpRunning)
{
    digitalWrite(RELAY_PIN, LOW);

    pumpRunning = true;

    Blynk.virtualWrite(V5, "PUMP ON");

    Serial.println("PUMP ON");
}

// -----
// Pump OFF
// -----

if (pumpOffHour >= 0 &&
    pumpOffMinute >= 0 &&
    now.hour() == pumpOffHour &&
    now.minute() == pumpOffMinute &&
    pumpRunning)
{
    digitalWrite(RELAY_PIN, HIGH);

    pumpRunning = false;

    Blynk.virtualWrite(V5, "PUMP OFF");
```



```
    Serial.println("PUMP OFF");
  }
}

// -----
// Setup
// -----

void setup()
{
  Serial.begin(115200);

  // RTC
  Wire.begin(5, 6);

  if (!rtc.begin())
  {
    Serial.println("RTC Not Found");

    while (1);
  }

  // Relay
  pinMode(RELAY_PIN, OUTPUT);

  digitalWrite(RELAY_PIN, HIGH);

  // MAX7219
  display.begin();
  display.setIntensity(3);

  // WiFi + Blynk
  Blynk.begin(
    BLYNK_AUTH_TOKEN,
    ssid,
    pass);

  timer.setInterval(
    1000L,
```

```
        updateClock);  
    }  
  
    // -----  
    // Loop  
    // -----  
  
void loop ()  
{  
    Blynk.run();  
    timer.run();  
}
```